

Subject: SAST Summer Training 2026, Agent Harness

Date: from 2026/10/12 to 2026/10/13

Contents

Tool Use	1
-----------------	----------

Tool Use

Tool 层概述

Tool 层把模型可见的能力描述、Harness 内部的安全策略和真正的执行函数封装在同一个接口中。模型只接触名称、提示和 Schema，而 Harness 还会使用权限、并发和结果映射等字段。

下面是经过简化的核心数据结构，包含 Tool、ToolResult 和 ToolUseContext 三部分。

```
// 工具池在一次 Query 中只读，避免执行期间被任意修改。
type Tools = readonly Tool[]

// Input: 输入 Schema 的类型；Output: 内部结果类型；P: 进度事件类型。
type Tool<
  Input extends AnyObject = AnyObject,
  Output = unknown,
  P extends ToolProgressData = ToolProgressData,
> = {
  // 模型调用工具时使用的稳定名称。
  name: string

  // 可选别名，用于兼容旧名称或不同入口的命名习惯。
  aliases?: string[]

  // 面向模型的能力描述与输入/输出协议。
  // description() 可以根据具体输入生成更有针对性的描述。
  description(
    input: z.infer<Input>,
    options: ToolDescriptionOptions,
  ): Promise<string>

  // prompt() 返回注入系统提示的工具使用说明。
  prompt(options: ToolPromptOptions): Promise<string>

  // inputSchema 在执行前校验模型生成的参数，并提供静态类型推导。
  readonly inputSchema: Input

  // outputSchema 可进一步校验工具实现返回的数据。
  outputSchema?: z.ZodType<unknown>

  // 面向 Harness 的能力开关、风险属性和权限检查。
  // isEnabled() 决定工具是否进入当前任务的可用工具池。
  isEnabled(): boolean

  // 只有不会互相干扰的调用才能并发执行。
  isConcurrencySafe(input: z.infer<Input>): boolean

  // 只读属性会影响默认权限策略和并发判断。
  isReadOnly(input: z.infer<Input>): boolean

  // 破坏性操作通常需要更严格的确认；并非所有工具都实现此方法。
  isDestructive?(input: z.infer<Input>): boolean

  // 可选的语义校验补充 Schema 无法表达的规则，例如路径是否存在。
  validateInput?(
```

```

        input: z.infer<Input>,
        context: ToolUseContext,
    ): Promise<ValidationResult>

    // 综合 allow/deny 规则、当前上下文和输入内容得到权限结果.
    checkPermissions(
        input: z.infer<Input>,
        context: ToolUseContext,
    ): Promise<PermissionResult>

    // Tool.call() 返回最终 ToolResult; 中间进度通过 onProgress 回调上报.
    call(
        // 已通过 Schema 校验的工具参数.
        input: z.infer<Input>,
        // 本次执行可访问的工具环境和任务状态.
        context: ToolUseContext,
        // 需要交互确认时调用的权限回调.
        canUseTool: CanUseToolFn,
        // 触发该工具的 assistant 消息, 用于建立消息关联.
        parentMessage: AssistantMessage,
        // 可选进度回调, 长任务可以多次调用.
        onProgress?: ToolCallProgress<P>,
    ): Promise<ToolResult<Output>>

    // 将内部输出映射为发回模型的 tool_result block.
    mapToolResultToToolResultBlockParam(
        // 工具内部使用的结构化结果.
        output: Output,
        // 必须与模型发出的 tool_use.id 一致.
        toolUseID: string,
    ): ToolResultBlockParam
}

type ToolResult<Output> = {
    // 工具的主要结构化输出.
    data: Output

    // 工具除标准结果外希望追加到会话中的消息.
    newMessages?: Message[]

    // 工具可返回一个纯函数, 更新后续工具调用使用的上下文.
    contextModifier?: (
        context: ToolUseContext,
    ) => ToolUseContext

    // MCP 工具可以保留协议定义的元数据和结构化内容.
    mcpMeta?: {
        _meta?: Record<string, unknown>
        structuredContent?: Record<string, unknown>
    }
}

type ToolUseContext = {
    // 创建 Query 时确定的工具、命令、模型和 Agent 配置.
    options: {
        // 本地斜杠命令或技能命令定义.
        commands: Command[]
        // 当前 Query 可以调用的完整工具池.

```

```

tools: Tools
// Agent Loop 默认使用的模型名称.
mainLoopModel: string
// 已连接的 MCP Server 客户端.
mcpClients: MCPServerConnection[]
// 可供主 Agent 委派子 Agent 定义.
agentDefinitions: AgentDefinitionsResult
}

// 用户取消任务时向正在运行的工具传播 AbortSignal.
abortController: AbortController

// 缓存已读取文件的状态, 用于检测执行期间发生的外部变化.
readFileState: FileStateCache

// 读取应用当前状态; 工具不应直接持有容易过期的状态副本.
getAppState(): AppState

// 通过更新函数安全地修改应用状态.
setAppState(update: (state: AppState) => AppState): void

// 存在 agentId 表示工具运行在子 Agent 上下文中.
agentId?: AgentId

// 当前调用 ID, 用于关联进度事件、权限请求和最终结果.
toolUseId?: string
}

```

BashTool 示例

BashTool 展示了一个具体工具如何实现 Schema、只读判断、权限匹配、命令执行、进度上报和结果映射。代码仍是教学简化版本。

```

// BashTool.tsx: 保留 Tool 定义、权限入口、进度和结果映射;
// UI、后台任务、平台兼容和错误恢复分支已折叠.
export const BashTool = buildTool({
  // name 是模型调用时使用的工具名; searchHint 用于工具检索或展示.
  name: BASH_TOOL_NAME,
  searchHint: 'execute shell commands',

  // 限制直接返回到上下文中的字符数, 避免超大输出挤占 Token 预算.
  maxResultSizeChars: 30_000,

  // strict 要求输入严格匹配 Schema, 减少模型产生多余字段的风险.
  strict: true,

  // getter 延迟构建 Schema, 便于根据运行环境生成定义.
  get inputSchema() {
    return inputSchema()
  },

  get outputSchema() {
    return outputSchema()
  },
})

```

```

// 只读命令可以安全并发；未知命令默认不可并发。
isConcurrencySafe(input) {
  return this.isReadOnly(input)
},

isReadOnly(input) {
  // cd 会改变后续命令的工作目录，因此需要参与只读安全分析。
  const hasCd = commandHasAnyCd(input.command)

  // 只有静态分析明确返回 allow，才把命令视为只读。
  return checkReadOnlyConstraints(input, hasCd).behavior === 'allow'
},

// 解析复合命令，为 Bash(git *) 等权限规则生成匹配器。
async preparePermissionMatcher({ command }) {
  // 安全解析器将管道或复合命令拆成独立的 argv 列表。
  const parsed = await parseForSecurity(command)

  // 无法可靠解析时采用保守匹配器，让后续权限层继续要求确认。
  if (parsed.kind !== 'simple') return () => true

  // 把 argv 重新组合成标准化字符串，避免直接匹配未经解析的原始文本。
  const subcommands = parsed.commands.map(
    command => command.argv.join(' '),
  )

  // 返回闭包，供权限引擎逐条测试 Bash(...) 规则。
  return pattern => {
    // 能提取固定前缀时使用精确/前缀匹配，否则回退到通配符匹配。
    const prefix = permissionRuleExtractPrefix(pattern)
    return subcommands.some(command =>
      prefix !== null
        ? command === prefix || command.startsWith(`${prefix} `)
        : matchWildcardPattern(pattern, command),
    )
  }
},

async checkPermissions(input, context) {
  // 汇总项目规则、用户规则、只读分析和沙箱配置，得到最终权限决定。
  return bashToolHasPermission(input, context)
},

async call(
  input,
  context,
  _canUseTool,
  parentMessage,
  onProgress,
) {
  // 某些 sed 编辑会被 Harness 转换为结构化文件补丁，
  // 这样可以绕过 Shell 字符串解析并获得更可靠的差异记录。
  if (input._simulatedSedEdit) {
    return applySedEdit(
      input._simulatedSedEdit,
      context,
      parentMessage,
    )
  }
}

```

```

}

// 创建异步命令执行器。此时只是获得生成器，命令进度由下面的 next() 驱动。
const command = runShellCommand({
  input,
  // 将任务取消信号传给子进程。
  abortController: context.abortController,
  // 后台任务使用专用状态更新器，否则使用普通应用状态更新器。
  setAppState:
    context.setAppStateForTasks ?? context.setAppState,
  setToolJSX: context.setToolJSX,
  // 子 Agent 不允许通过 cd 永久改变共享工作目录。
  preventCwdChanges: Boolean(context.agentId),
  isMainThread: !context.agentId,
  // 两个 ID 用来把进程状态关联回正确的调用和 Agent。
  toolUseId: context.toolUseId,
  agentId: context.agentId,
})

// runShellCommand() 是 AsyncGenerator:
// yield 值是 Bash progress, return 值是最终 ExecResult.
let next
do {
  // 每次 next() 要么得到一条进度，要么得到 done: true 的最终返回值。
  next = await command.next()
  if (!next.done) {
    // 可选链保证没有订阅进度时也能继续执行命令。
    onProgress?.({
      toolUseID: context.toolUseId,
      data: {
        type: 'bash_progress',
        output: next.value.output,
        elapsedTimeSeconds: next.value.elapsedTimeSeconds,
      },
    })
  }
} while (!next.done)

// 真实源码会在这里累计 stdout、处理 stderr、退出码、中断状态、
// 图片输出和超大输出持久化，构造 BashTool 的 Out 数据。
// buildBashOutput() 是本讲义的教学化辅助名称，不是源码函数名。
const data = buildBashOutput(input, next.value, context)

// ToolResult 使用对象包装 data，以便未来附加 newMessages 或
// contextModifier.
return { data }
},

// Tool 执行输出最终在这里变成 Anthropic tool_result block.
mapToolResultToToolResultBlockParam(result, toolUseID) {
  return {
    type: 'tool_result',
    // 此 ID 必须对应原始 tool_use，否则模型无法建立调用与结果的关系。
    tool_use_id: toolUseID,
    // 去掉空输出后合并 stdout 和 stderr，形成模型可阅读的 observation.
    content: [result.stdout, result.stderr]
      .filter(Boolean)
      .join('\n'),
  }
}

```

```
// 中断被标记为错误结果，但仍作为合法 tool_result 返回模型。  
is_error: result.interrupted,  
    }  
},  
)
```